

La programmation orientée objet en PHP

ENSA de Tanger

introduction

Nous abordons l'une des notions les plus importantes en programmation. Il s'agit en fait de la Programmation Orientée Objets, mieux connue sous l'acronyme de POO. Ce style de programmation s'appuie sur le concept d'objets. Ce cours suppose que vous possédez déjà une certaine base en POO.

Pourquoi la POO ?

- * La **POO** est utilisé dans les projets conséquent, elle permet la gestion des systèmes complexes et est très modulables. On la trouve dans les frameworks comme CakePHP, Symfony, Prada ou Zend Framework,...
- * Une **classe** est un ensemble qui peut contenir des données et des fonctions. Une classe est un espace clos et sécurisé elle rassemble des données et des fonctionnalités qui sont étroitement liés.

Classes et objets

- * On parlera de classe pour définir l'**implémentation** du code PHP, c'est une représentation abstraite d'un objet.
- * Une **classe** définit une structure de base pour un possible objet : propriétés et méthodes y seront déclarées.
- * Un **objet** est une **instance** de la classe, c'est une variable que l'on peut utiliser pour appeler les données et les fonctionnalités de la classe. On peut créer autant d'objet que l'on veut à partir d'une seule et même classe.

Classe (définition)

- * Une classe définit une structure de base pour un possible objet : propriétés et méthodes y seront déclarées.
- * Une classe est définie par un ensemble de méthodes et d'attributs, que l'on appelle également les membres d'une classe. On définit dans le code une classe à l'aide du mot clef **class**. Les attributs auront plus généralement une visibilité privée et les méthodes seront, plus généralement, publiques.
- * **Par convention** : on commencera le nom des classes par une majuscule.

```
<?php  
class Chaise{  
    ...  
} ?>
```

De bonnes pratiques tout de suite

- * Lors de la création d'une classe on utilisera le principe suivant : « **une classe un rôle** », par exemple la classe pour se connecter à la base de données, la classe pour gérer les droits des utilisateurs de l'application.
- * Dans une classe on peut également appeler d'autres objets. On peut également faire du polymorphisme. On parlera dans ce cas de couplage pour désigner ces liaisons. Il faudra donc appliquer le principe suivant : peu de liaison entre les classes : **couplage faible**.

Objet (définition)

- * **Un objet** est une variable possédant une structure complexe constituée de propriétés (variables) et de méthodes (fonctions). En effet, les variables contenues dans un objet sont nommées attributs et les fonctions, méthodes.
- * **Un objet** est une représentation concrète de la notion de classe. On crée un objet à partir d'une classe via une instantiation à l'aide du mot clef : **new**

Une instantiation est en fait une sorte de copie de la structure à l'intérieur d'une variable. En résumé, vous insérer dans une variable votre classe afin de pouvoir accéder à ses méthodes et attributs.

```
<?php
    $monVelo = new Velo() ;
    $unAutreVelo = new Velo() ;
?>
```

Instanciación

Une instanciación est en fait une sorte de copie de la structure à l'intérieur d'une variable. En résumé, vous insérer dans une variable votre classe afin de pouvoir accéder à ses méthodes et attributs.

Exemple

```
<?php
// Début d'une classe
class Uneclasse {
    //...
}
//Instanciacion de la classe
$objet= new Uneclasse();
?>
```

Les méthodes

Une **méthode** est, comme mentionné plus haut, un ensemble d'instructions PHP. Il vous permet d'effectuer des opérations précises sur les attributs de votre classe ou exécuter des instructions quelconques.

L'accès aux méthodes (**fonctions**) d'un objet se fait sensiblement de la même manière que pour les attributs. Il suffit d'écrire le nom de la variable contenant l'objet instancié, suivi d'une flèche (->) ; c'est à dire le signe moins (-) et le signe supérieur (>), et du nom de la méthode. La méthode doit impérativement être suivie des parenthèses, contenant les arguments, s'il y en a.

Les attributs

Un **attribut** ou **propriété** est une variable à l'intérieur d'un objet. Ces attributs sont propres à votre objet et ne seront pas influencés par des modifications faites sur un autre objet identique.

L'accès aux attributs se fait de la même manière que pour les méthodes. Cependant, vous ne devez pas inclure de signe de dollar (\$) devant le nom de votre **propriété**.

Accéder aux attributs

Un attribut est, comme mentionné plus haut, une variable à l'intérieur d'un objet. Pour accéder aux attributs d'un objet, il vous faut écrire la variable contenant une instanciation de l'objet, suivi d'une flèche (->) ; c'est à dire le signe moins (-) et le signe supérieur (>), suivi du nom de l'attribut en question (**sans le signe \$**).

Exemple

```
<?php
class Organe{
private $_cellule ;
private $_tissus;
public $couleur ;
}

$organe = new Organe() ; // l'objet
$organe->couleur; // un attribut de l'objet
```

Accéder aux attributs (suite)

Pour accéder aux attributs à **l'intérieur** même de la classe, il suffit simplement d'utiliser le mot-clé `$this`, suivi d'une flèche et du nom de l'attribut. Le mot-clé `$this` indique que vous faites référence à l'objet en cours, donc la classe.

Exemple

```
<?php
class Velo {
private $_couleur = "rouge";
// méthode : ObtenirCouleur
public function ObtenirCouleur(){
    echo 'Je suis de couleur ' . $this->_couleur;
}
}
$monVelo = new Velo ; // création de l'objet
echo $monVelo->ObtenirCouleur() ; //appel de la méthode
echo $monVelo->_Couleur; //erreur
?>
```

Les constructeurs

Un constructeur est en fait une méthode qui sera automatiquement appelée lors de l'instanciation de l'objet. Il vous permet d'attribuer des valeurs par défaut, effectuer des connexions, démarrer une session, etc.

Les objets

Un objet peut également, à partir de PHP5, posséder des méthodes réservées au constructeur et destructeur. Ces nouvelles notions seront approfondies un peu plus loin dans ce cours.



Example 1

Nouveautés

Cette partie contient des notions disponibles **uniquement à partir de PHP5** et ne fonctionneront donc **pas** avec les versions antérieures

Constructeur/Destructeur

L'une des principales nouveautés dans la POO avec PHP est l'introduction de méthodes réservées au constructeur et destructeur. Un objet se construit, s'utilise puis se détruit. Ces méthodes réservées servent donc à construire et détruire l'objet.

Le constructeur

Lors de l'instanciation de l'objet, la méthode réservée `__construct()` est automatiquement appelée. Cependant, dans un souci de compatibilité descendante, l'ancienne méthode est toujours supportée. Si la méthode `__construct()` n'est pas définie, PHP recherchera un constructeur portant le même nom que la classe.

Le constructeur (suite)

L'un des principaux avantages de cette nouvelle méthode réservée, ce que vous n'aurez plus à renommer votre constructeur advenant le fait que vous auriez à renommer votre classe.

Exemple

```
<?php
```

```
// Définition d'une classe
```

```
class Uneclasse {  
    public $propriete;  
    public function __construct() {  
        $this->propriete = 'Valeur';  
    }  
}
```

```
// Instanciation de la classe
```

```
$objet = new uneClasse();  
echo $objet->propriete;  
?>
```

Destructeur

PHP5 introduit le destructeur de classe. Ce destructeur se charge d'exécuter certaines opérations lors de la destruction de l'objet. Il vous permet de fermer une connexion ou un fichier automatiquement à la fin de l'exécution d'un script.

Destructeur (suite)

La méthode réservée `__destruct()`, utilisée pour définir un destructeur, sera automatiquement appelée à la fin de votre script par le gestionnaire de mémoire de PHP lors de la vidange des mémoires.

Vous pouvez librement appeler le destructeur à l'intérieur de vos scripts, cependant **la mémoire attribuée à votre objet ne sera libérée qu'à la fin de votre script.**



Exemple 2



Visibilité

Public/Protected/Private

PHP5 introduit une nouvelle notion, celle de la visibilité. Certaines méthodes ou propriétés peuvent être rendues inaccessible de l'extérieur, et ainsi protéger certaines méthodes ou propriétés réservées à certains traitements internes.

Public

Le préfixe `public` définit une méthode ou propriété publique accessible à tous les niveaux, soit à partir de la classe, d'un descendant ou de l'extérieur.

Afin d'assurer une compatibilité descendante, les méthodes et propriétés déclarées à l'aide de l'ancienne méthode seront considérées comme `public`. Cependant, il est fortement recommandé d'utiliser `public` à `var` en PHP5.

Exemple

```
<?php
// Définition d'une classe
class uneClasse {
    public $propriete;
    function methode() {
    }
    public function methode2() {
    }
}
?>
```

Protected

Le préfixe *protected* définit une méthode ou propriété privée accessible qu'à partir de la classe elle-même et ses descendantes.

Toutes tentatives d'accès à ces membres protégés résulteront par une erreur fatale.



Exemple 3

Private

Le préfixe `private` définit une méthode ou propriété privée accessible qu'à partir de la classe mère.

Toutes tentatives d'accès à ces membres protégés à partir de l'extérieur ou d'un descendant, résulteront par une erreur fatale.



Exemple 4

Surcharge (Overloading)

La surcharge en PHP permet de créer dynamiquement des membres et des méthodes. Ces entités dynamiques sont traitées via des méthodes magiques établies dans une classe pour divers types d'actions.

Les méthodes surchargées sont appelées lors de l'interaction avec les membres et les méthodes qui n'ont pas été déclarés ou ne sont pas visibles dans le contexte courant.

PHP5 introduit trois nouvelles méthodes réservées : `__get()`, `__set()` et `__call()`.

Ces nouvelles méthodes permettent de gérer des appels à des méthodes ou propriétés non définies dans la classe.

Les méthodes réservées `__get()` et `__set()`

`__get()` et `__set()` sont utilisées pour les appels de **propriétés (attributs)**.

`__get()` possède un **paramètre** qui est le nom de la propriétés demandées.

`__set()` possède deux paramètres qui sont le **nom** de la propriétés demandées ainsi que la **valeur** que l'on désire lui attribuer.



Exemple 5

La méthode réservée `__call()`

`__call()` est utilisée pour les appels de méthodes. `__call()` possède deux paramètres qui sont le nom de la méthode demandée ainsi qu'un tableau des paramètres passés lors de son appel.

`__call` (string *\$name* , array *\$arguments*)

L'argument *\$name* est le nom de la méthode appelée. L'argument *\$arguments* est un tableau contenant les paramètres passés à la méthode *\$name*.



Exemple 6



Les interfaces en PHP5

Introduction

Contrairement à PHP4, PHP5 permet de gérer les interfaces. Elles permettent de créer un modèle que les classes qui l'implémentent doivent respecter. Les interfaces sont très pratiques lorsque plusieurs personnes développent un même projet, ou lorsqu'on développe un site modulaire.

Les interfaces

- * Les interfaces objet vous permettent de créer du code qui spécifie quelles méthodes et variables une classe peut implémenter, sans avoir à définir comment ces méthodes seront gérées.
- * Les interfaces sont définies en utilisant le mot clé interface, de la même façon qu'une classe standard mais sans aucun contenu de méthode.
- * Toutes les méthodes déclarées dans une interface doivent être publiques.

implements

Pour implémenter une interface, l'opérateur **implements** est utilisé. Toutes les méthodes de l'interface doivent être implémentées dans une classe ; si ce n'est pas le cas, une erreur fatale sera émise. Les classes peuvent implémenter plus d'une interface en séparant chaque interface par une virgule.



Exemple 7



L'opérateur de résolution de portée (::)

Présentation

L'opérateur de résolution de portée, appelé le symbole "double deux points" (::) (« Scope Resolution Operator » en anglais), fournit un moyen d'accéder aux membres statiques ou constants ainsi qu'aux éléments redéfinis par la classe. Il est utilisé pour appeler des éléments appartenant à telle classe et non à tel objet. Lorsque vous référencez ces éléments en dehors de la définition de la classe, utilisez le nom de la classe.

Static

Le fait de déclarer des membres ou des méthodes comme statiques vous permet d'y accéder sans avoir besoin d'instancier la classe. **Un membre déclaré comme statique ne peut être accédé avec l'objet instancié d'une classe** (bien qu'une méthode statique le peut).

La déclaration **static** doit être faite après la déclaration de visibilité.

Static

Comme les méthodes statiques sont appelables sans instance d'objet créée, la pseudo variable **\$this** n'est pas disponible dans la méthode déclarée en tant que statique.

En fait, les appels de méthodes statiques sont résolus au moment de la compilation. Lorsque l'on utilise un nom de classe explicite, la méthode est déjà identifiée complètement et aucune notion d'héritage n'est appliquée. Si l'appel est effectuée par le mot clé **self**, alors self est traduit en la classe courante, qui est la classe appartenant au code. Ici aussi, aucune notion d'héritage n'est appliquée.

Static

Depuis PHP 5.3.0, il est possible de référencer la classe en utilisant une variable. La valeur de la variable ne peut être un mot clé (e.g. *self*, *parent* et *static*).

Définition et utilisation d'une constante de classe

```
<?php
class MyClass
{
    const CONSTANT = 'valeur constante';

    function showConstant() {
        echo self::CONSTANT . "\n";
    }
}

echo MyClass::CONSTANT . "\n";

$classname = "MyClass";
echo $classname::CONSTANT . "\n"; // Depuis PHP 5.3.0

$class = new MyClass();
$class->showConstant();

echo $class::CONSTANT . "\n"; // Depuis PHP 5.3.0
?>
```



Exemple 8



Constantes de classe

Présentation

Il est possible de définir des valeurs constantes à l'intérieur d'une classe, qui ne seront pas modifiables. Les constantes diffèrent des variables normales du fait qu'on n'utilise pas le symbole \$ pour les déclarer ou les utiliser. Tout comme pour les membres statiques, on ne peut pas accéder aux valeurs constantes depuis une instance de l'objet (en utilisant `$object::constant`).



Exemple 9



Abstraction de classes

Présentation

PHP 5 introduit les classes et les méthodes abstraites. Il n'est pas autorisé de créer une instance d'une classe définie comme abstraite. Toutes les classes contenant au moins une méthode abstraite doivent également être abstraites. Pour définir une méthode abstraite, il faut simplement déclarer la signature de la méthode et ne fournir aucune implémentation.



Exemple 10

N.B.

Dans la classe qui **dérive** d'une classe abstraite, vous devez définir les modificateurs d'accessibilité des méthodes avec une **visibilité égale ou plus large** que celle de la méthode abstraite.



Méthode et classe finales

Méthode et classe finales

La création de **sous-classes** et la **redéfinition** des méthodes sont les éléments essentiels de la POO.

Il peut cependant être nécessaire d'empêcher toute modification d'une méthode ou d'une classe, en particulier dans un projet où l'on travaille en équipe.

Pour interdire la redéfinition d'une méthode d'une classe parente dans ses classes dérivées, faits précéder le mot-clé **function** du mot-clé **final**.



Exemple 11

classe finale

De même, pour interdire l'héritage d'une classe parente dans une classe enfant, faites précéder le mot-clé class du mot-clé final. Si vous définissez la classe suivante :

et si vous tentez de la dériver en créant la classe enfant suivante :

Fatal error: Class triangleiso may not inherit from final class (triangle)

```
final class triangle {  
    //Instructions  
}
```

```
class triangleiso extends triangle{  
    //Instructions  
}
```



Clonage d'objet

Clonage d'objet

La notion de clonage d'objet est une des nouveautés introduites dans PHP 5. Elle permet d'effectuer une copie exacte d'un objet mais en lui affectant une zone de mémoire différente de celle de l'objet original. Contrairement à la création d'une simple copie à l'aide de l'opérateur = ou d'une référence sur un objet avec l'opérateur &, les modifications opérées sur l'objet cloné ne sont pas répercutées sur l'original.

Clonage d'objet (suite)

Pour cloner un objet, utilisez le mot-clé **clone** selon la syntaxe suivante :

```
$objetclone = clone $objet ;
```

L'objet cloné a exactement les mêmes propriétés et les mêmes méthodes que l'original. Après le clonage, les modifications opérées sur la variable **\$objet** n'ont aucun effet sur le clone **\$objetclone**, et réciproquement.



Parcours d'objets

Présentation

- * PHP 5 fournit une façon de définir les objets de manière à ce qu'on puisse parcourir une liste de membres avec une structure foreach . Par défaut, toutes les propriétés visibles seront utilisées pour le parcours.



Exemple 12

Les exceptions

PHP a une gestion des exceptions similaire à ce qu'offrent les autres langages de programmation. Une exception peut être lancée ("**throw**") et attrapée ("**catch**") dans PHP.

Le code devra être entouré d'un bloc **try** pour faciliter la saisie d'une exception potentielle. Chaque **try** doit avoir au moins un bloc **catch** correspondant. Plusieurs blocs **catch** peuvent être utilisés pour attraper différentes classes d'exceptions.

Exemple

```
1 <?php
2 function inverse($x) {
3     if (!$x) {
4         throw new Exception('Division par zéro.');
```

5 }

```
6     return 1/$x;
7 }
8
9 try {
10     echo inverse(5) . "<br/>";
11     echo inverse(0) . "<br/>";
12 } catch (Exception $e) {
13     echo 'Exception reçue : ', $e->getMessage(), "<br/>";
14 }
15
16 // Continue execution
17 echo "Bonjour le monde !<br/>";
18 ?>
```



PDO

Qu'est-ce que PDO

L'extension **PHP Data Objects** (PDO) définit une excellente interface pour accéder à une base de données depuis PHP.

Ce qui signifie que vous utilisez les mêmes fonctions pour exécuter des requêtes ou récupérer les données quelque soit la base de données utilisé

Quelle utilité ?

L'hétérogénéité des moyens d'accès aux bases de données a été un des problèmes les plus gênants en PHP.

Le problème vient du fait que, par exemple, les fonctions permettant l'accès à une base **PostgreSQL** sont différentes de celles pour **MySQL**, et elles sont toutes différentes de celles pour **SQLite**, etc.

Quelle utilité ?

L'hétérogénéité des moyens d'accès aux bases de données a été un des problèmes les plus gênants en PHP.

Le problème vient du fait que, par exemple, les fonctions permettant l'accès à une base **PostgreSQL** sont différentes de celles pour **MySQL**, et elles sont toutes différentes de celles pour **SQLite**, etc.

Se connecter à la base de données

Les connexions sont établies en créant des instances de la classe de base de PDO.

Il suffit pour cela d'exécuter la commande :

```
$dbh = new  
PDO("type_de_base:host=machine_qui_héberge_la_base;dbname=nom_  
de_la_base", "nom_d_utilisateur", "mot_de_passe") ;
```

Le type de base

Le `type_de_base` quant à lui est une valeur parmi :

- * `mssql` (FreeTDS/Microsoft SQL Server/Sybase),
- * `firebird` (Firebird/Interbase 6),
- * `informix` (IBM Informix Dynamic Server),
- * `mysql` (MySQL 3.x/4.x/5.x),
- * `oci` (Oracle Call Interface),
- * `odbc` (ODBC v3 --IBM DB2 unixODBC et win32 ODBC--)),
- * `pgsql` (PostgreSQL),
- * `sqlite` (SQLite 3 et SQLite 2).

Exécuter une requête

En ayant auparavant obtenu un identifiant de connexion `$dbh` (instance de la classe de base de PDO) comme précédemment, la syntaxe est simple :

```
$res = $dbh->query(votre_requête);
```

Exemple de requête de type SELECT :

```
$result = $dbh->query('SELECT id FROM table') ;  
echo $result->rowCount() ; // nombre de lignes  
echo $result->columnCount() ; // nombre de champs
```

Accéder aux résultats d'une requête select

Accès par le nom des colonnes :

```
$result = $dbh->query("SELECT * FROM villes");  
$data = $result->fetchAll();  
foreach ( $data as $row ) {  
    echo $row["codepostal"]." - ".$row["ville"]."<br />\n";  
}
```

Accéder aux résultats d'une requête select

Accès par le numéro des colonnes :

```
$result = $dbh->query("SELECT * FROM villes");  
$data = $result->fetchAll();  
foreach ( $data as $row ) {  
    for ($i=0 ; $i<$result->columnCount() ; $i++) {  
        echo $row[$i]." ** ";  
    }  
    echo "<br />\n";  
}
```



Exemple

myparam.inc.php

```
1  <?php
2  define ("MYHOST", "localhost");
3  define ("MYUSER", "root");
4  define ("MYPASS", "");
5  ?>
```

connexpdo.inc.php

```
1 <?php
2 function connexpdo($base,$param)
3 {
4     include_once($param.".inc.php");
5     $dsn="mysql:host=".MYHOST.";dbname=".$base;
6     $user=MYUSER;
7     $pass=MYPASS;
8     try
9     {
10        $idcom = new PDO($dsn,$user,$pass);
11        return $idcom;
12    }
13    catch(PDOException $except) //PDOException Représente une erreur émise par PDO
14    {
15        echo"Echec de la connexion",$except->getMessage();
16        return FALSE;
17        exit();
18    }
19 }
20 ?>
```

Remarque

Chacun de vos scripts d'accès à la base doit de ce fait contenir les lignes suivantes :

- * `include("connexpdo.inc.php")`
- * `$idcom = connexpdo ("nom_base","myparam ");`

L'opération de connexion est donc gérée en 2 lignes, quel que soit le script ou le type de base.

Envoi de requêtes SQL au serveur

Pour envoyer une requête au serveur, nous avons le choix entre plusieurs méthodes.

- * Pour celles qui ne retournent pas de résultats, il existe la méthode `exec()` des objets PDO dont la syntaxe est la suivante :

`integer $idcom->exec(string requete)`

Elle est utilisée pour les requêtes INSERT, UPDATE ou DELETE par exemple. Elle retourne simplement un entier qui correspond au nombre de lignes concernées par la requête.

Envoi de requêtes SQL au serveur

- * Pour les requêtes qui vont retourner des résultats, il faut employer la méthode `query()`, dont la syntaxe est :

`object $idcom->query(string $requete)`

Elle retourne `FALSE` en cas d'erreur ou, sinon, un objet de la classe `PDOStatement` représentant l'ensemble des lignes de résultats pour lesquelles il faut ensuite appeler les méthodes spécialisées pour afficher les valeurs.



Exemple 13

Exécuter une requête sans retour

C'est une requête qui ne retourne pas de données.

Par exemple, les requêtes qui suppriment, modifient ou insèrent des données.

```
<?php require 'sqlconnect.php';  
    $sql = 'DELETE FROM accouns WHERE type = "member";'  
    $nb = $pdo->exec($sql);  
    echo $nb.' membres ont été supprimés.';  
?>
```

Récupérer des données

```
1. <?php
2. require 'sqlconnect.php';
3.
4. $sql = 'SELECT * FROM membres';
5. $req = $pdo->query($sql);
6. while($row = $req->fetch()) {
7.     echo '<a href="membre-'. $row['id'] .'.html">'. $row['pseudo'] . '</a><br/>';
8. }
9. $req->closeCursor();
10. ?>
```


Astuce

Pour protéger vos requêtes contre les injections SQL avec la méthode **quote()**, je vous propose de regarder cet exemple :

```
$sql = 'SELECT * FROM membres WHERE pseudo = '.$pdo->quote($pseudoMembre
```

Requêtes préparées

La plupart des bases de données supportent le concept des **requêtes préparées**.

Qu'est-ce donc ? Vous pouvez les voir comme une sorte de modèle compilé pour le SQL que vous voulez exécuter, qui peut être personnalisé en utilisant des variables en guise de paramètres.

Requêtes préparées

Les requêtes préparées offrent deux fonctionnalités essentielles :

- * La requête ne doit être analysée (ou préparée) qu'une seule fois, mais peut être exécutée plusieurs fois avec des paramètres identiques ou différents.
- * Les paramètres pour préparer les requêtes n'ont pas besoin d'être entre guillemets ; le driver le gère pour vous.

Exemple #1 Insertions répétitives en utilisant les requêtes préparées

Cet exemple effectue une requête INSERT en y substituant un *nom* et une *valeur* pour les marqueurs nommés.

```
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (:name, :value)");
$stmt->bindParam(':name', $name);
$stmt->bindParam(':value', $value);
```

```
// insertion d'une ligne
```

```
$name = 'one';
$value = 1;
$stmt->execute();
```

```
// insertion d'une autre ligne avec des valeurs différentes
```

```
$name = 'two';
$value = 2;
$stmt->execute();
```

```
?>
```

Exemple #2 Insertions répétées en utilisant des requêtes réparées

Cet exemple effectue une requête INSERT en y substituant un *nom* et une *valeur* pour les marqueurs ?.

```
<?php
$stmt = $dbh->prepare("INSERT INTO REGISTRY (name, value) VALUES (?, ?)");
$stmt->bindParam(1, $name);
$stmt->bindParam(2, $value);

// insertion d'une ligne
$name = 'one';
$value = 1;
$stmt->execute();

// insertion d'une autre ligne avec différentes valeurs
$name = 'two';
$value = 2;
$stmt->execute();
?>
```

Exemple #3 Récupération des données en utilisant des requêtes préparées

Cet exemple récupère des données basées sur la valeur d'une clé fournie par un formulaire. L'entrée utilisateur est automatiquement échappée, il n'y a donc aucun risque d'attaque par injection SQL.

```
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY where name = ?");
if ($stmt->execute(array($_GET['name']))) {
    while ($row = $stmt->fetch()) {
        print_r($row);
    }
}
?>
```



Le modèle MVC et le contrôleur sous PHP

Principe du modèle MVC

Le but de MVC est de séparer une application en parties (en 3 parties distinctes, au minimum, et le plus souvent). On va donc distinguer :

1. **Le modèle** qui encapsule la logique métier et la manipulation des sources de données.
2. **La vue** : présente les données à l'utilisateur.
3. **Le contrôleur** : c'est tout le reste, c'est lui qui analyse la requête client, accède aux données, formate le tout, et balance le tout à la partie vue, qui va afficher ça.

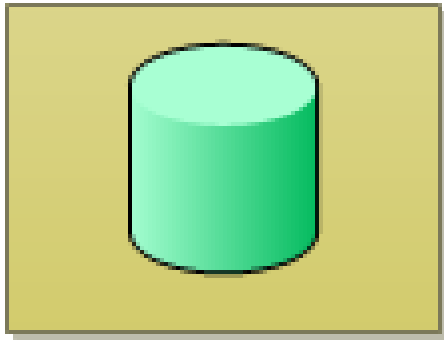
modèle MVC

Et voilà, on a appliqué le modèle MVC.

On a bien séparé le code en 3 parties distinctes, qui se connaissent l'une l'autre. Dans ce code-là, on peut facilement changer la manière d'accéder aux données (changer de SGBD, par exemple), sans pour autant se soucier le moins du monde de la présentation, ou de la manipulation future de ces données.

Et nous n'avons pas utilisé la POO

schéma des rôles de chacun de ces éléments



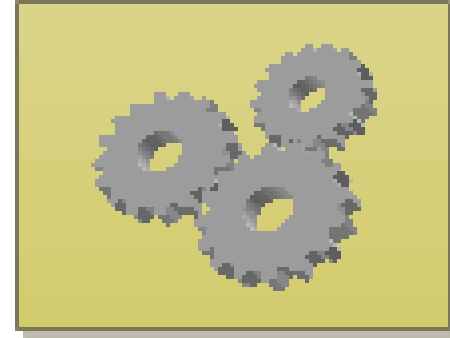
Modèle

(accès à la base de données)



Vue

(affichage de la page)



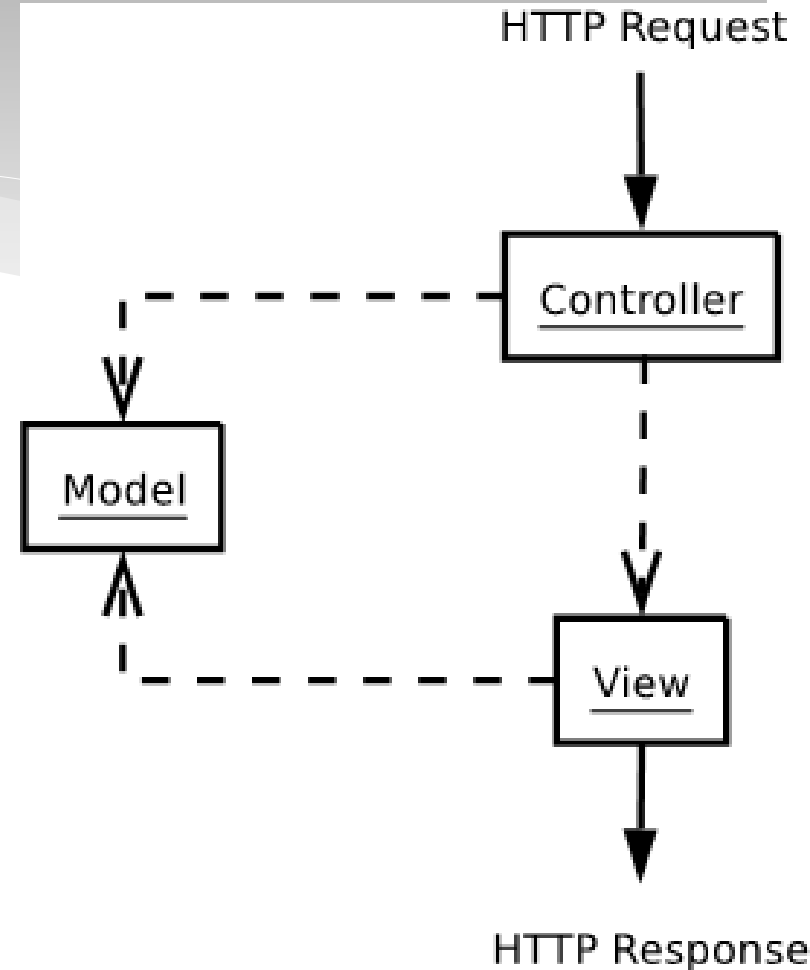
Contrôleur

(logique, calculs et décisions)

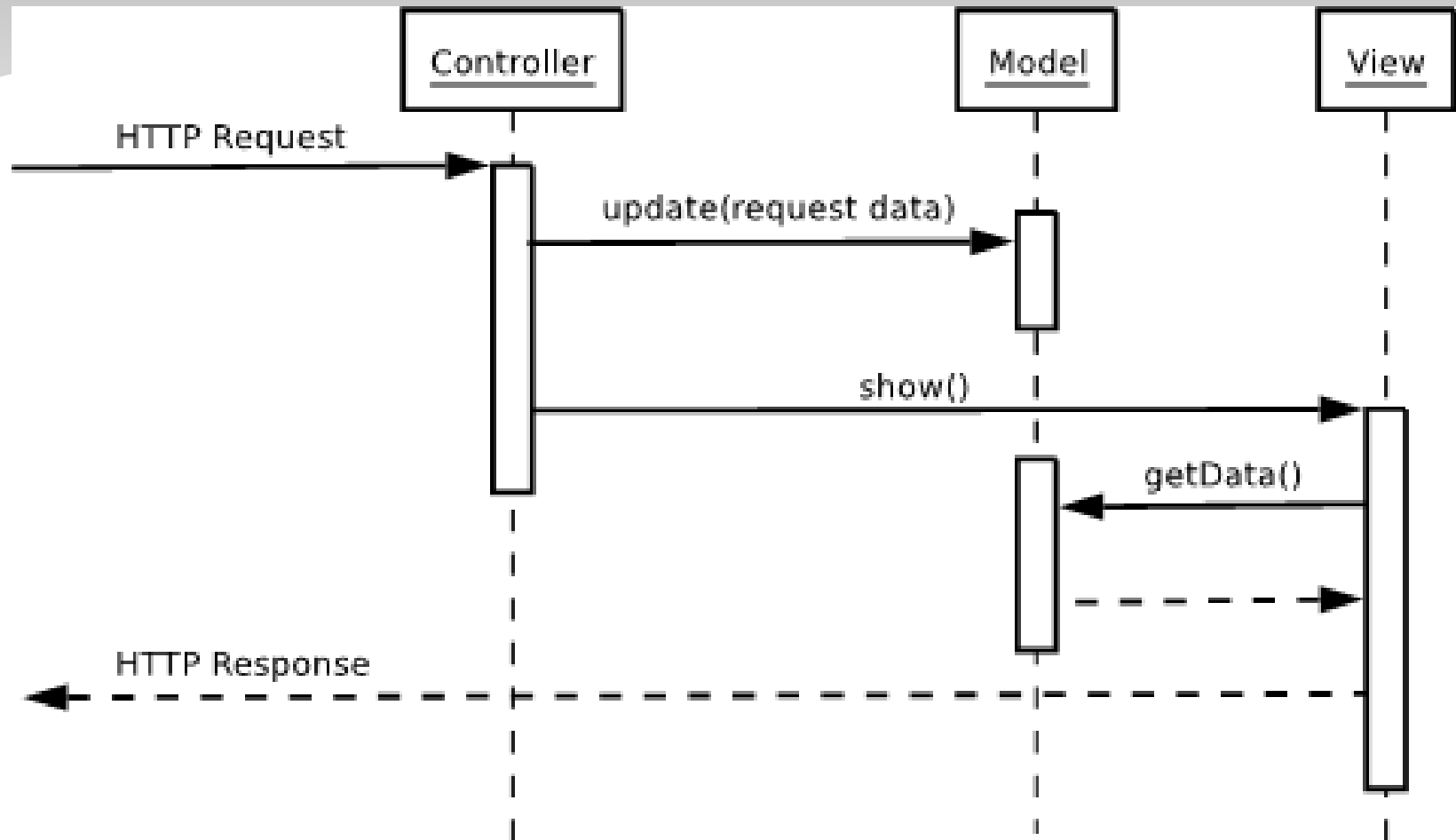
L'intérêt du modèle MVC

Le modèle MVC décrit une manière d'architecturer d'une application informatique en la décomposant en trois sous-parties de manière à ce que plusieurs groupes de personnes puissent bosser chacun sur leur partie, sans même qu'ils connaissent les personnes qui bossent les autres parties :

- la partie Modèle ;
- la partie Vue ;
- la partie Contrôleur.



modèle MVC



la vue

la vue d'abord, c'est un `TemplateView`. On utilise PHP lui-même pour écrire nos variables dans du HTML, ou alors un moteur de template.

Il est très important de noter que la vue ne fait que présenter des données, c'est le modèle qui va fournir toutes les méthodes nécessaires à la présentation. Si par exemple on doit afficher en gras les commentaires datant de moins de 3 jours, le modèle fournira une méthode d'accès à ces commentaires, et la vue comportera une condition de vérification.

Le modèle

Du côté du modèle, il est très important que celui-ci ne connaisse ni les contrôleurs, ni les vues. Les liens vont des contrôleurs vers le modèle, de temps en temps, des vues vers le modèle, mais jamais dans le sens inverse.

Le modèle étant totalement indépendant du reste, il pourra faire l'objet d'une réflexion et d'un développement à part du reste. Il pourra de même être utilisé par d'autres composants, tels que des webservices.

Le contrôleur

Et on arrive enfin au contrôleur, c'est lui le composant principal du modèle MVC. Il agit avec le modèle, la vue, et d'autres contrôleurs, à la fois.

Le contrôleur doit donc :

1. Analyser la requête, soit extraire les informations dans l'URL
2. Faire une mise à jour du modèle si nécessaire, en lui passant des paramètres
3. Déterminer la vue à afficher et demander son affichage

Le contrôleur (suite)

Chaque contrôleur regroupe l'ensemble des responsabilités dans un seul fichier, appelé dans l'URL, c'est le motif le plus clair et le plus facile à mettre en œuvre, la POO n'étant pas nécessaire.

Cependant dans des applications qui gagnent un peu en complexité, de la factorisation s'impose, afin de séparer certains rôles :

Le **FrontController**, ou contrôleur frontal, est un motif qui est destiné à prendre en charge l'analyse de la requête client. Il est donc le seul et unique point d'entrée de l'application, et va orchestrer toute la suite. C'est un singleton et généralement on l'appelle index.php et on lui fournit les actions demandées dans l'URL :

http://monsite.com/index.php?action=show_livres&categorie=x